

Software Requirements Specification For Appointment Reminder and Preparation System

By: Ajani Douglas, Nikola Konjikusic and Satjal Pant

Date Made: February 7, 2026

Course: Software Requirements Engineering

Professor: Sadit Khan

Table Of Contents:

Section 1 - Introduction	3
1.1 Purpose:	3
1.2 Intended Audience and Reading Suggestions:	3
1.4 Project Scope	4
Section 2 - Overall Description	5
2.1 Product Perspective	5
2.2 Product features (Functions):	5
2.3 User Classes and Characteristics	6
2.4 Operating Environment	6
Section 3	7
3.1 User Interfaces	7
3.2 Hardware Interfaces	7
3.3 Software Interfaces	7
3.4 Communications Interfaces	8
Stakeholders	9
Interview Questions	10
Non - Functional Requirements:	11
Functional Requirements:	13
Use Cases	15
Section 4	17
4.1 Selected High-Priority Use Case	17
4.2 Formal Use Case Description	17
4.3 Swim Lane Activity Diagram	18
4.4 Use Case Diagram – All Use Cases (Deliverable #3)	19
Appendix E	20
CRC Cards	20
Class Diagram	22
GenAI output	23
State Diagrams	27
Party Analysis	29

Section 1 - Introduction

1.1 Purpose:

This new software product will address the problem of users keeping track of their appointments by sending timely reminders and providing useful preparation information before each scheduled appointment, making it easier for users to organize and prepare.

1.2 Intended Audience and Reading

Suggestions:

This document is intended for stakeholders involved in the Appointment Reminder and Preparation System.

Developers -should focus on functional and system design.

Project Managers - should review the introductions, scope, and project goals.

Testers (QA) - should study the requirements to create test cases and verify system accuracy.

End Users - should read the system overview and user instructions to understand how to use the system.

System Administrators - should focus on security, configuration, and maintenance.

Document Writers and Stakeholders - should review the full document to prepare manuals and understand system value.

1.4 Project Scope

People can forget things; this is especially true of appointments, whether it's because they're too busy and it slips their mind, or they end up double-booking themselves. However, with our mobile app, the goal is to lessen the stress surrounding appointments. Users can keep track of how many appointments they have to attend and prevent committing to too many at once, as it would inform them of any double bookings if needed. The user could set the time, where they need to be, what the appointment is and what they need to bring, whilst setting the appointment up in the app, and in turn the app will send reminders of everything through push notifications, so the user doesn't forget these things over time, in addition to keeping track of their appointments. In the app, all of this will be available to view using a calendar format.

Section 2 - Overall Description

2.1 Product Perspective

The Appointment Reminder and Preparation is a new, self-contained software designed to manage appointments, send automated reminders, and provide preparation instructions. It helps reduce missed appointments and improve communication.

The system can interface with existing scheduling systems and external services such as email and SMS platforms to send notifications. Main components include the user interface, application, logic and data.

2.2 Product features (Functions):

The new software will provide the following functionalities:

1. Create, update, and delete appointments
2. Schedule reminder notifications
3. Add preparation checklists for appointments
4. Send push notifications, messages and/or emails with appointment details such as date, time, and location
5. Create an account that allows the user to access their appointments across different devices

2.3 User Classes and Characteristics

Clients / Patients / Customers - Receive appointment reminders and preparation instructions. They may have basic technical skills and access the system through mobile devices or email.

Staff / Receptionists / Service Providers - Schedule appointments, update client information, and monitor reminder delivery. They are familiar with basic computer system.

System Administrators - Manage user accounts, system settings, security, and performs system maintenance such as password resets and backups.

Managers / Supervisors - View reports and system performance to monitor efficiency and missed appointments.

Students - Use the system to receive appointment reminders for classes, advising sessions, or meetings, schoolwork/assignments, and they can view preparation instructions, confirm or cancel appointments, and receive notifications through email, SMS, or app alerts. Students typically access the system using smartphones, tablets, or laptops/desktops.

2.4 Operating Environment

The app:

Will use the Nylas Cloud Calendar API.

Will support both IOS and Android.

Will be developed with JavaScript and Swift.

Section 3

3.1 User Interfaces

The Appointment Reminder and Preparation System provides a mobile interface for iOS and Android devices. The interface follows standard mobile design conventions and supports straightforward navigation.

Users can also view appointments in a calendar format, create and manage said appointments, and configure reminder settings. Login and registration screens allow users to access their accounts. Push notifications display essential appointment details such as title, date, time, and location. System feedback messages are brief and direct.

3.2 Hardware Interfaces

The system operates on standard iOS and Android smartphones and tablets. User interaction occurs through touchscreen input. Notifications are delivered using the device's native notification system. No specialized hardware is required.

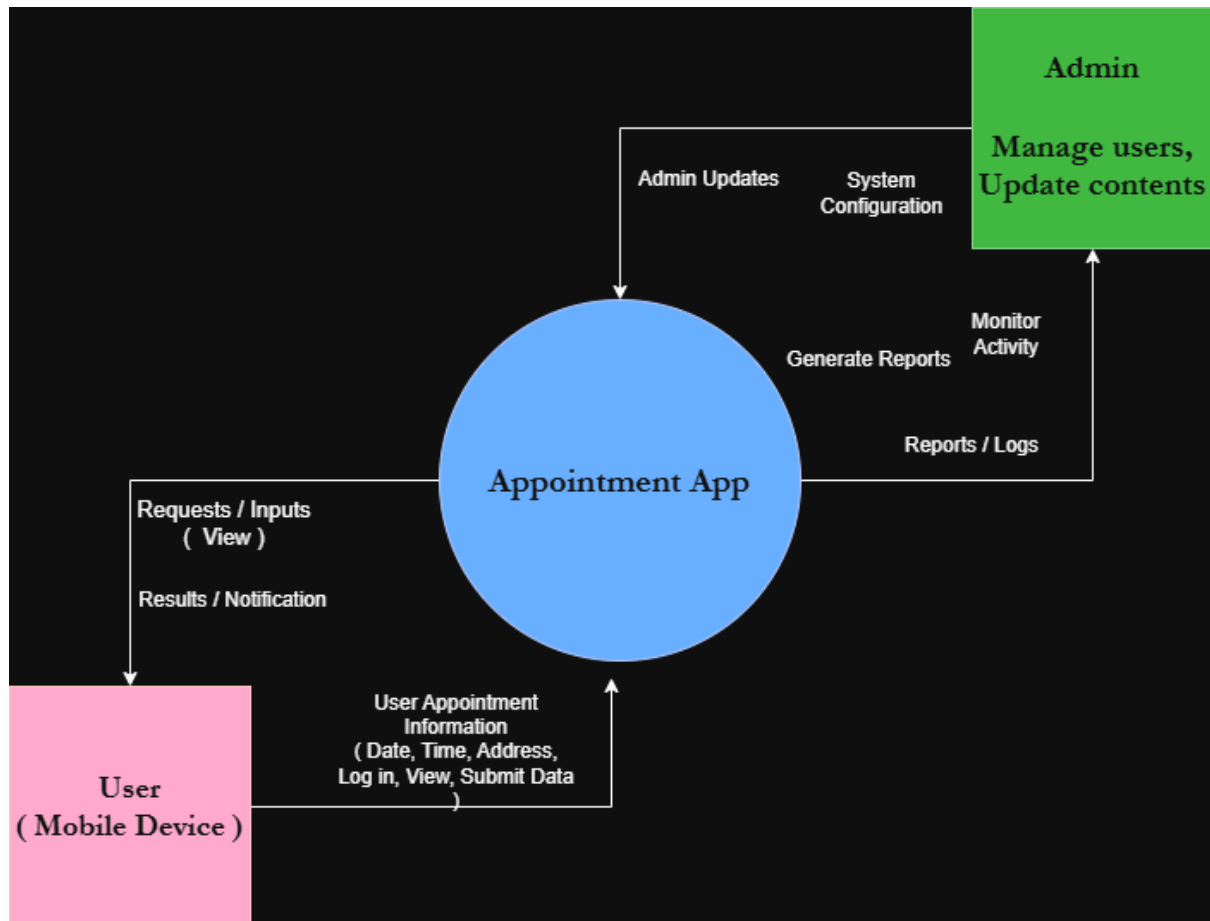
3.3 Software Interfaces

The system interfaces with the Nylas Cloud Calendar API for appointment scheduling and synchronization. Additionally, it interacts with mobile operating systems and backend services responsible for authentication, data storage, and reminder delivery.

Data is exchanged using JSON format. Access to stored data is limited to authorized users.

3.4 Communications Interfaces

The system uses HTTPS for secure communication between the application and backend services. Reminder delivery is supported through push notifications, with optional email and SMS services. All transmitted data is encrypted to protect user information.



Stakeholders

Stakeholder Name	Stakeholder Position	External/Internal	Stakeholder contact details	Operational/Executive	Interest
Quinn Moore	Investor	External	quinnmoore@yahoo.com	Executive	medium
Brooke Holden	Customer	External	holdenbr@gmail.com	Operational	high
Helena Robinson	Programmer	Internal	helena.r@fastservice.club	Operational	medium
Johnathan Downs	Supplier	External	jdowns84@mailhaven.xyz	Executive	low
Daisy Boone	Marketing Manager	Internal	daisyboone@openmail.life	Executive	medium
Russell Wells	Marketing Officer	Internal	wellrussell@wavenmail.site	Operational	low
Darren Cox	IT Administrator	Internal	darrencox@inboxloop.net	Operational	medium
Haileigh Simpson	UI Designer	Internal	haileigh.simpson@privaterelay.app	Operational	high
Thomas Farell	Data Analyst	Internal	t.farell@securepost.work	Executive	low
Ezekiel Odom	IT Security Officer	Internal	zodom@freesecond.space	Executive	medium

Interview Questions

Question	Stakeholder Position	Answer
1) What problems do you currently face when managing multiple appointments?	Customer	I forget appointments, double-book myself, or forget where or when i'm going.
2) What features would make appointment reminders most useful to users?	Marketing Manager	Custom reminder timing and customizable notifications would increase user satisfaction.
3) What security concerns should be considered when storing appointment data?	IT Security Officer	User data should be protected with secure logins and encrypted data storage.
4) What system qualities are most important to ensure reliability	IT Administrator	High uptime and fast notification delivery are critical to avoid missed reminders.
5) What technical considerations are important when developing this system?	Programmer	The system should support API integration, data synchronization, and be scalable for future updates.
6) What design considerations are important for user adoption	UI Designer	The interface should be easy to navigate and consistent across iOS and Android

Non - Functional Requirements:

Requirement ID	Requirement Title	Description	Priority	Requestor
-----	----- -	-----	-----	-----
NFR01	System Speed	The app should send reminders close to the time the user expects them. Notifications should not feel delayed, even when many people are using the app at once.	High	IT Administrator
NFR02	Data Protection	User information should be kept private and secure. Logins should be protected, and appointment details should not be accessible to anyone who is not authorized.	High	IT Security Officer
NFR03	System Uptime	The app should be available most of the time and not crash often. If something	High	IT Administrator

		goes wrong, the system should recover without deleting appointments or reminders.		
NFR04	Device Support	The app should work properly on both iOS and Android phones. Users should not lose features or reminders because of the type of device they are using.	Medium	UI Designer
NFR05	Ease of Use	The app should be easy to understand, even for users who are not very technical. Basic actions like checking appointments or setting reminders should feel simple and straightforward.	Medium	Customer
NFR06	Future Growth	The system should be able to handle more users and appointments in the future without needing to be	Low	Investor

		rebuilt from scratch.		
--	--	-----------------------	--	--

Functional Requirements:

Requirement ID	Requirement title	Short Description	Priority	Requester
FR01	Appointment Overview	The app should provide an overview of all appointments that have been set for a customizable time frame. (a week, two weeks, maximum a month)	High	Customer
FR02	Register	The app should allow users to register to create an account with their email and a password.	High	Customer
FR03	Login	The app should allow users to log in to their accounts to access their appointments	High	Customer
FR04	Customizable Alerts	Users should be allowed to able to set when they get alerts(a day before, a few hours before), what is in the alerts(what the appointment is, what they need to bring), and where they get them(email, SMS, push notifications)	High	Customer
FR05	Countdown	The app should allow users to view a countdown to appointments, and should also allow them to pin it as a notification.	Low	Customer
FR06	Data Sync	The app should be able to sync an account's	High	Customer

		appointments across different devices so users can access their reminders regardless of what device they're using.		
FR07	Create Appointment	The user should be able to create appointments and set their date and time, whatever they need to bring, whether or not it should be reoccurring and the address	High	Investor
FR08	Edit Appointment	The user should be able to edit appointments after they've been created, and they should be able to change any details in case anything about the appointment changes.	High	Investor
FR09	Export and Import Events	The user should be able to export their appointment list as a CSV file and be able to import the appointment list into other calendars. The user should also be able to have other calendars' events imported into it.	High	Investor
FR10	Multi-Language Support	The system should support multiple languages, so that the user can set whatever language they're comfortable with.	High	Customer

Use Cases

Use Case name	Requirements ID	Actor(s)	Brief Description
View Appointment Overview	FR01	Customer	When the actor first opens the app, the app will immediately display an overview of all appointments for a time period they can set themselves. The default will be a week's timespan
Customize alerts	FR04	Customer	The actor will go into settings, then they will click the customizable alerts option, from here, they will be given choices to receive alerts through SMS, Email and Push Notifications. They will also be given choices to set the number of alerts received and the amount of time between each alert.
Enable Countdown	FR05	Customer	The actor will go into settings, and one of the options will ask them if they would like to enable countdowns for their appointments. Upon selecting yes, the actor will receive a notification that begins counting down until the appointment, similar to the timer notification with the clock app.
Sync Data	FR06	Customer	The actor will log in to an account with data already on it. The software will check if the account already has data associated with it from another device, and if there is, it will prompt the actor to choose between syncing the data between the two devices or starting fresh.
Create Appointment	FR07	Customer	The actor will press the plus icon on the overview screen, which will take them to a screen which prompts them for all the appointment details. From there, the actor will enter the details, click save, and the appointment will be saved.
Edit Appointment	FR08	Customer	The actor will tap on one of their appointments and press the pencil icon, which allows them to edit all the details of the appointment. Once the actor has made

			all their edits, they click save, and all the new details will be saved.
Import Appointments	FR09	Customer	The actor will open settings and click the option to import from another app. From there, the actor will select a CSV file from a calendar to import into the app. From there, the app will read the file and create appointments from any events in the file for the user to edit.
Choose language	FR10	Customer	When opening the app, the login and create account screens will have an option in the corner to change the language
Create an account	FR02		When the app is first opened on a new device, the user will be given the choice between logging in and registering. When the user chooses to register, the software will prompt them for their email, their phone number and a password. Once these are filled out, the software will check whether the phone number and email are already in use and whether the entered password is secure. If these requirements are fulfilled, the account will be created.
Log into account	FR03		When the app is first opened on a new device, the user will be given the choice between logging in and registering. When the user chooses to log in, the software will prompt for the user's email or phone number and the corresponding password. The software will then check if the password is correct. If it is, the user will be logged into the account; if it isn't, then the user will be prompted to try again.

Section 4

4.1 Selected High-Priority Use Case

Requirement ID: FR07

Use Case Name: Create Appointment

Priority: High

Rationale for Selection: The “Create Appointment” functionality is the foundational use case of the Appointment Reminder and Preparation System. The Primary value of the software is sending reminders, preventing double-bookings, and tracking preparation checklists. Relies entirely on the successful execution of this use case.

4.2 Formal Use Case Description

Use Case Name: Create Appointment

Scenario: Customer creates a new standard appointment with a preparation checklist and automated reminders.

Triggering Event: The Customer selects the “Add Appointment” option from the application dashboard or calendar view.

Brief Description: The Customer enters the details for a new appointment, including date, time, location, and a “what to bring” checklist. The system verifies the data, checks for double-bookings, saves the appointment locally, synchronises it across devices via the Nylas API, and schedules the requested reminder alerts.

Actors: Customer

Related Use Cases: Sync Data (FR06), Customize Alerts (FR04)

Stakeholders:

- Customer: Wants the appointment saved accurately to receive timely reminders.
- IT Administrator / System: Requires the data to be formatted correctly for database storage and secure syncing.

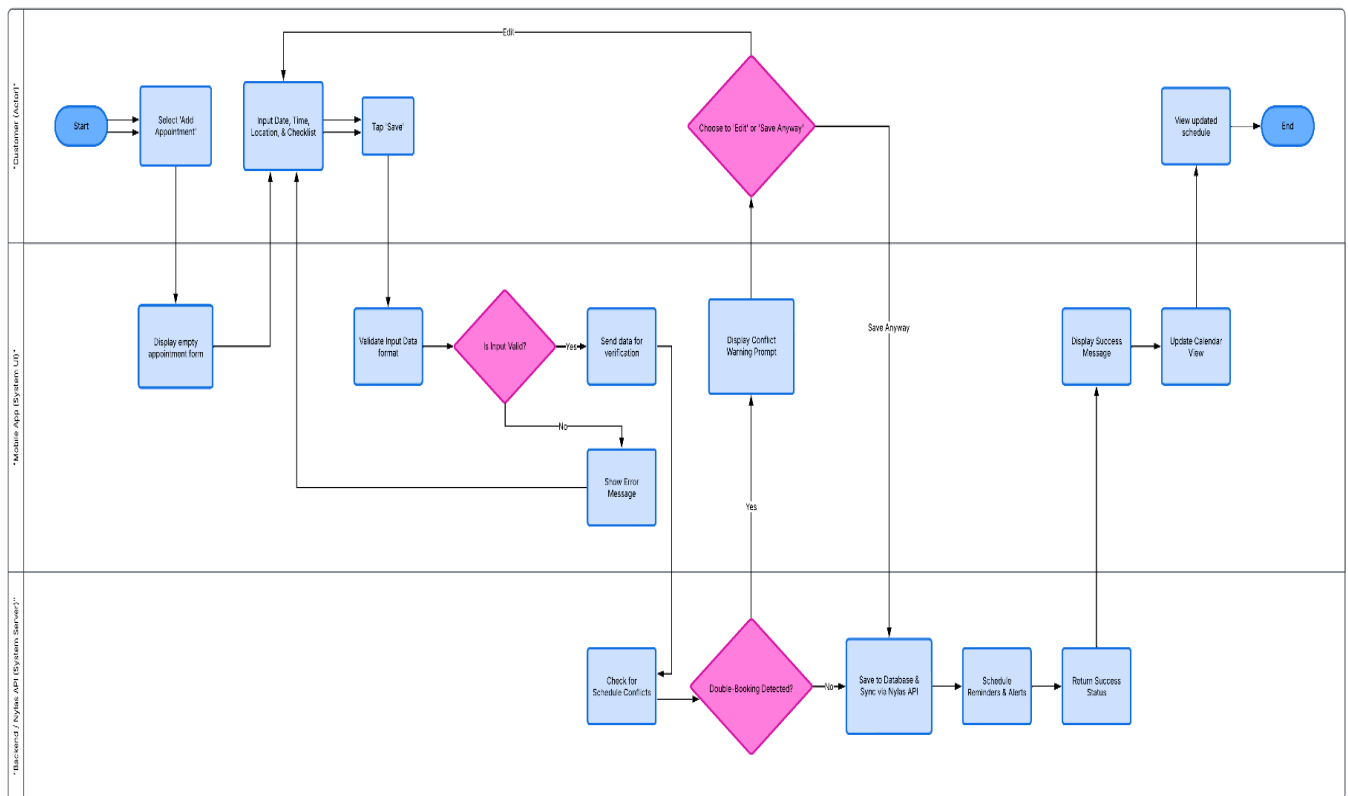
Preconditions:

1. The Customer has downloaded the app, successfully registered, and is logged into their account.
2. The device has an active internet connection (for cloud synchronization).

Postconditions:

1. The new appointment is saved in the system database.
2. The appointment is synchronized across the user's devices.
3. Push notifications/reminders are scheduled based on system defaults or user preferences.

4.3 Swim Lane Activity Diagram



4.4 Use Case Diagram – All Use Cases (Deliverable #3)

The use case diagram below groups all identified use cases by subsystem: Appointment Management Module, Notification & Reminder Module, and User Account Module.

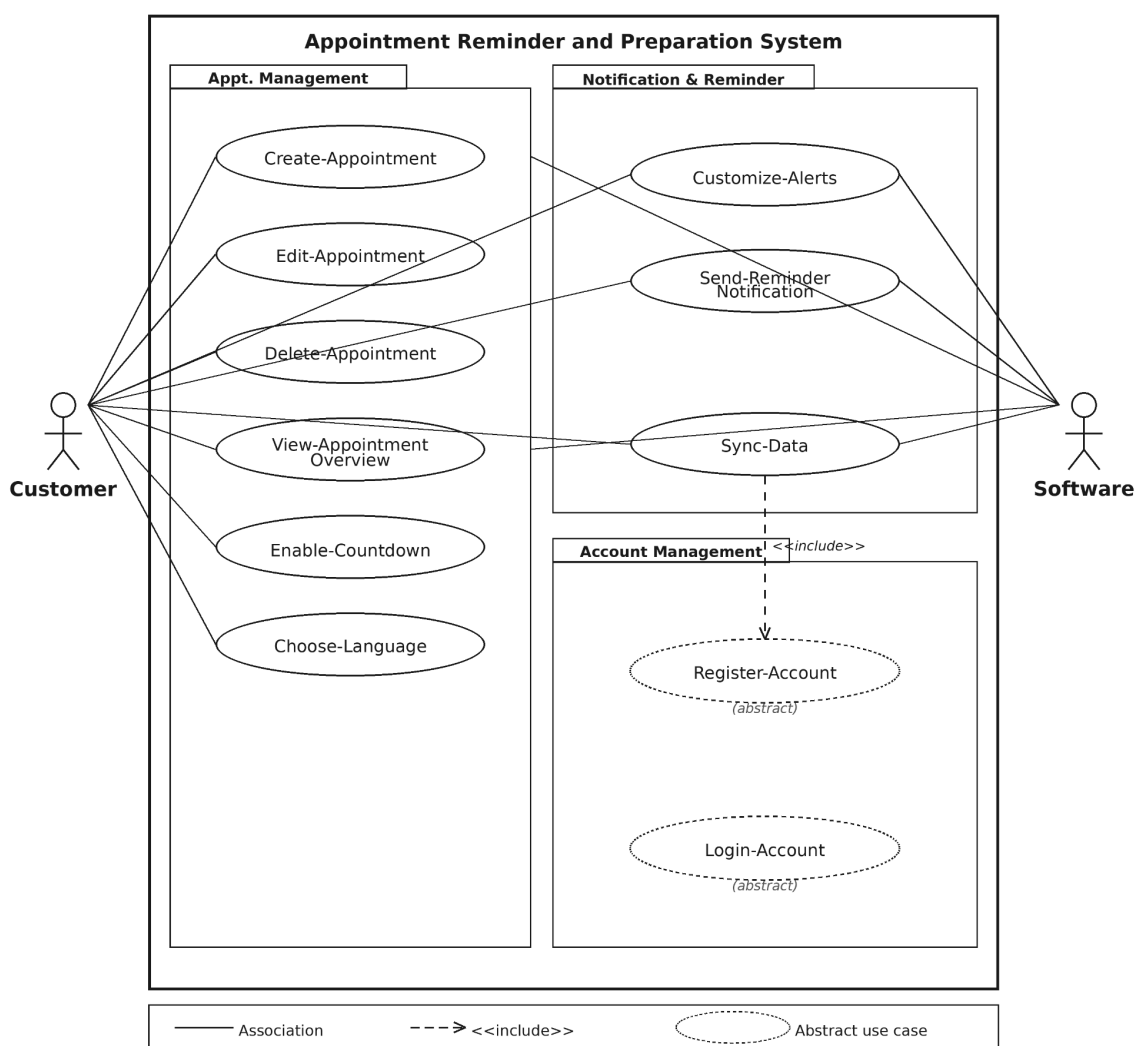


Figure: Use Case Diagram – Appointment Reminder and Preparation System

This diagram maps out how users and the system interact with the Appointment Reminder app. The Customer can manage appointments, set reminders, and customize alerts, while the Software handles the notification and syncing side of things. Abstract use cases like Login and Register sit in the background, supporting the rest.

Appendix E

CRC Cards

User	
Responsibility	Collaboration
Create/Edit/Delete Appointments	Appointment
Provide input for appointments and preferences	Reminder
View Appointments	Settings
Receive Notifications/Reminders	Authentication
Change Language	
Change Password	

Settings	
Responsibility	Collaboration

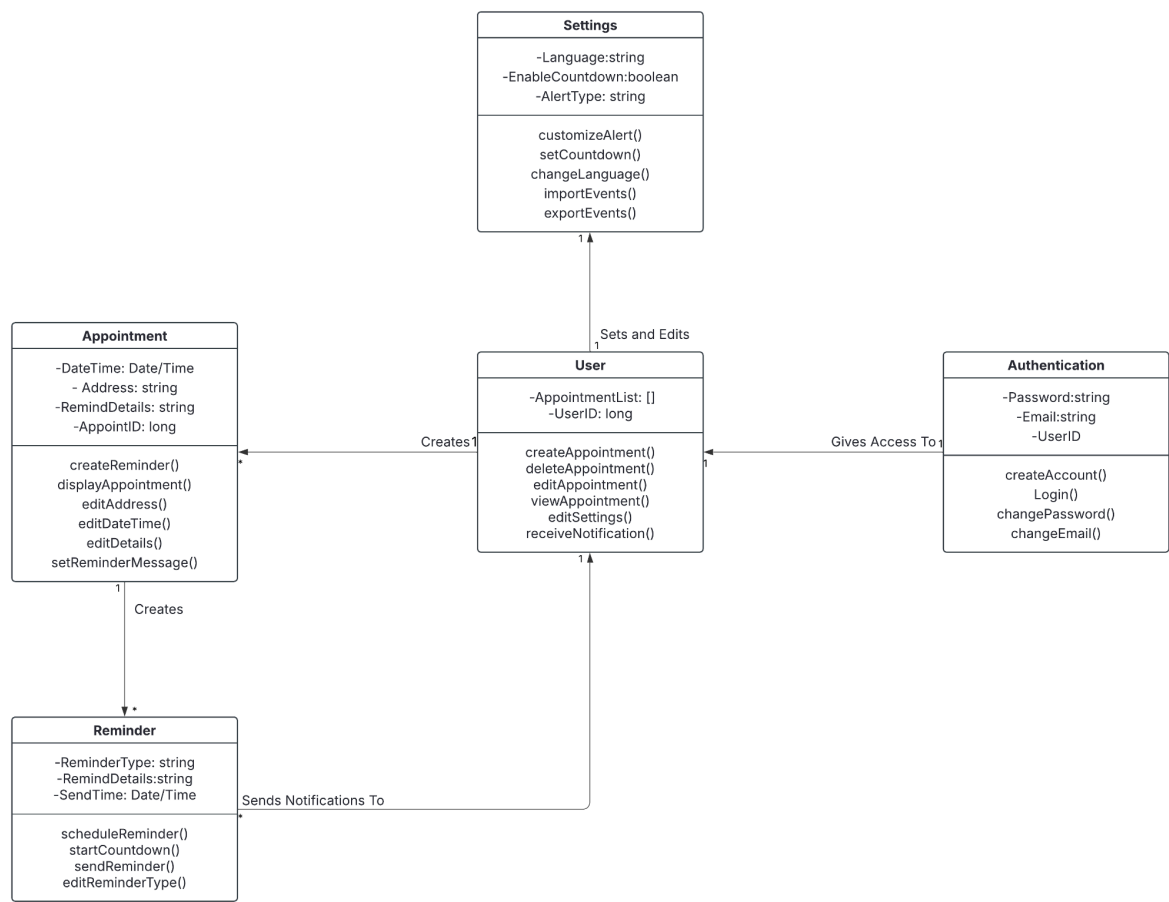
Settings	
Customize Alerts	User
Set Countdown on and off	
Change Language	
Import/Export Events	

Authentication	
Responsibility	Collaboration
Create Accounts	User
Login	
Change Password	
Change Email	

Reminder	
Responsibility	Collaboration
Send Reminder	Appointment
Set Reminder Type	User
Schedule Reminder	
Start Countdown	

Appointment	
Responsibility	Collaboration
Create Reminders	User
Display Appointment	Reminder
Edit Details	
Delete Appointment	

Class Diagram



GenAI output

```
from dataclasses import dataclass, field
from datetime import datetime
from typing import List, Optional

# -----
# Settings
# -----
@dataclass
class Settings:
    language: str = "English"
    notifications_enabled: bool = True
    alert_type: str = "popup"

    def customize_alert(self, alert_type: str):
        self.alert_type = alert_type

    def set_countdown(self, enabled: bool):
        self.notifications_enabled = enabled

    def change_language(self, language: str):
        self.language = language

    def import_events(self, events: list):
        print(f"Imported {len(events)} events")

    def export_events(self) -> list:
        print("Exporting events")
        return []

# -----
# Reminder
# -----
```

```

@dataclass
class Reminder:
    reminder_type: str
    reminder_details: str
    send_time: datetime

    def schedule_reminder(self):
        print(f"Reminder scheduled at {self.send_time}")

    def start_countdown(self):
        print("Countdown started")

    def send_alarm(self):
        print("Alarm sent")

    def edit_reminder_type(self, reminder_type: str):
        self.reminder_type = reminder_type

```

```

# -----

```

```

# Appointment

```

```

# -----

```

```

@dataclass
class Appointment:
    date_time: datetime
    address: str
    reminder_details: str
    appointment_id: int
    reminder: Optional[Reminder] = None

    def create_reminder(self, reminder: Reminder):
        self.reminder = reminder

    def display_appointment(self):
        print(f"Appointment #{self.appointment_id}")
        print(f" Address: {self.address}")

```



```
print(f" Date/Time: {self.date_time}")
print(f" Details: {self.reminder_details}")
```

```
def set_address(self, address: str):
    self.address = address
```

```
def set_date_time(self, date_time: datetime):
    self.date_time = date_time
```

```
def edit_details(self, details: str):
    self.reminder_details = details
```

```
def set_reminder_message(self, message: str):
    self.reminder_details = message
```

```
# -----
```

```
# Authentication
```

```
# -----
```

```
@dataclass
```

```
class Authentication:
```

```
    email: str
    password: str
    user_id: int
```

```
    def create_account(self):
        print(f"Account created for {self.email}")
```

```
    def login(self, email: str, password: str) -> bool:
        return self.email == email and self.password == password
```

```
    def change_password(self, new_password: str):
        self.password = new_password
```

```
    def change_email(self, new_email: str):
        self.email = new_email
```

```

# -----
# User
# -----
@dataclass
class User:
    user_id: int
    appointments: List[Appointment] = field(default_factory=list)
    settings: Optional[Settings] = None
    authentication: Optional[Authentication] = None

    def create_appointment(self, appointment: Appointment):
        self.appointments.append(appointment)

    def delete_appointment(self, appointment_id: int):
        self.appointments = [
            a for a in self.appointments if a.appointment_id != appointment_id
        ]

    def edit_appointment(self, appointment_id: int, **changes):
        for appointment in self.appointments:
            if appointment.appointment_id == appointment_id:
                for key, value in changes.items():
                    if hasattr(appointment, key):
                        setattr(appointment, key, value)

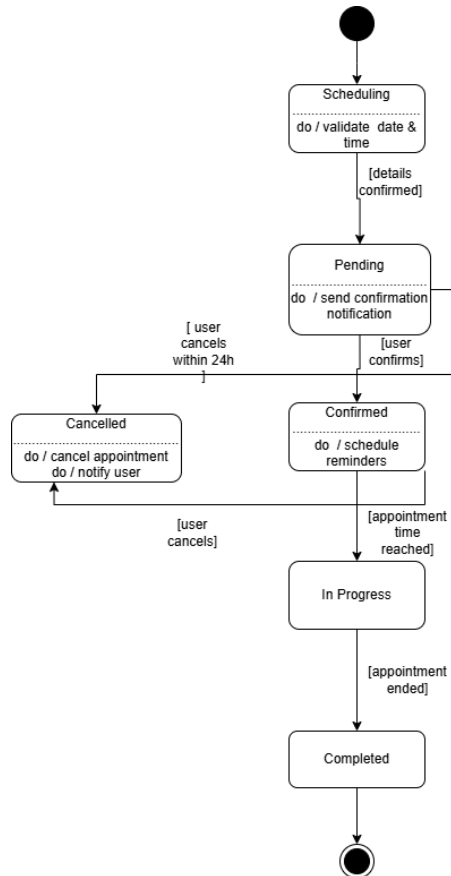
    def view_appointments(self):
        for appointment in self.appointments:
            appointment.display_appointment()

    def receive_notification(self, message: str):
        print(f'Notification for user {self.user_id}: {message}')

```

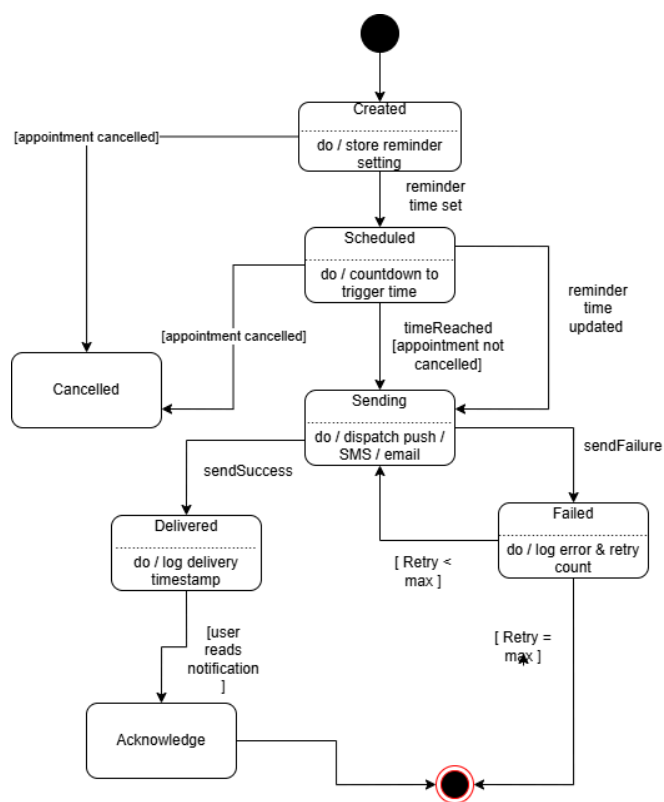
State Diagrams

Appointment :



This state diagram models the lifecycle of an Appointment object. It begins in the **Scheduling** state, where the system validates the date and time. Once details are confirmed, the appointment moves to **Pending**, where a confirmation notification is sent. If the user confirms, it transitions to **Confirmed**, which schedules reminders. When the appointment time is reached, it enters **In Progress**, and upon completion moves to **Completed**, reaching the end state. At any point before completion, the user may cancel — transitioning to the **Cancelled** state, with a guard condition of [user cancels within 24h] applied from the **Pending** stage.

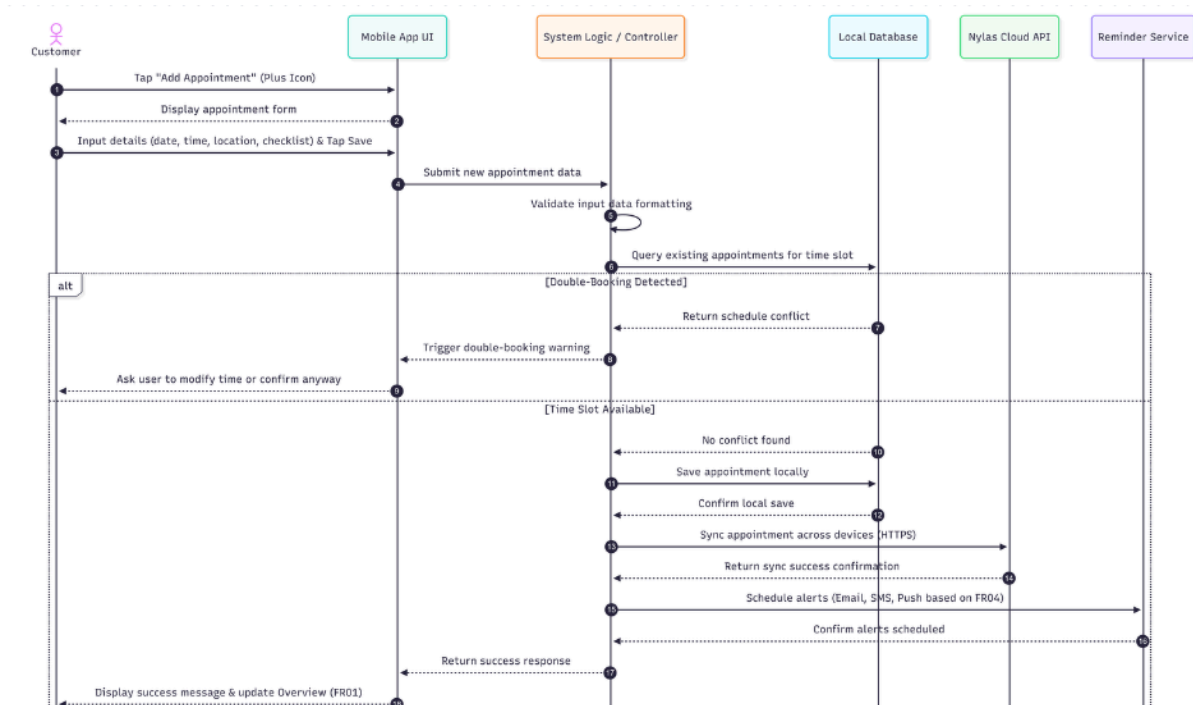
Reminder :



This state diagram models the lifecycle of a Reminder object. Starting in the **Created** state, reminder settings are stored and it moves to **Scheduled**, where it counts down to the trigger time. If the appointment is cancelled, the reminder transitions directly to **Cancelled**. When the trigger time is reached [appointment not cancelled], it enters **Sending**, dispatching a push/SMS/email notification. On success, it moves to **Delivered**, where the timestamp is logged, then to **Acknowledge** once the user reads the notification, before reaching the end state. If delivery fails, it enters **Failed** — if [Retry < max], it loops back to **Sending**; if [Retry =

max], it terminates. A self-loop on Scheduled handles cases where the reminder time is updated.

Sequence Diagram



GenAi Sequence

```
import json
import sqlite3
from datetime import datetime
from typing import List, Dict, Optional, Any
from dataclasses import dataclass
from enum import Enum
import smtplib
import ssl
import requests
from dataclasses import dataclass, field
from datetime import datetime, timedelta
```

=====

```
# DATA MODELS
```

```
# =====
```

```
@dataclass
```

```
class Appointment:
```

```
    """Represents an appointment entity"""
```

```
    date: str # Format: YYYY-MM-DD
```

```
    time: str # Format: HH:MM (24-hour)
```

```
    location: str
```

```
    checklist: List[str] # List of checklist items
```

```
    appointment_id: Optional[int] = None
```

```
    def to_dict(self) -> Dict:
```

```
        return {
```

```
            'id': self.appointment_id,
```

```
            'date': self.date,
```

```
            'time': self.time,
```

```
            'location': self.location,
```

```
            'checklist': json.dumps(self.checklist)
```

```
        }
```

```
    @classmethod
```

```
    def from_dict(cls, data: Dict) -> 'Appointment':
```

```
        checklist = json.loads(data['checklist']) if isinstance(data['checklist'], str) else  
data['checklist']
```

```
        return cls(
```

```
            appointment_id=data.get('id'),
```

```
            date=data['date'],
```

```
            time=data['time'],
```

```
            location=data['location'],
```

```
            checklist=checklist
```

```
        )
```

```
class AlertType(Enum):
```

```
    EMAIL = "email"
```

```
SMS = "sms"
PUSH = "push"
```

```
@dataclass
```

```
class AlertConfig:
```

```
    """Alert configuration for an appointment"""
```

```
    appointment_id: int
```

```
    alert_type: AlertType
```

```
    recipient: str # email address or phone number
```

```
    minutes_before: int = 30 # Send alert X minutes before appointment
```

```
# =====
# LOCAL DATABASE MANAGER
# =====
```

```
class LocalDatabase:
```

```
    """Manages local SQLite database for appointments"""
```

```
    def __init__(self, db_path: str = "appointments.db"):
```

```
        self.db_path = db_path
```

```
        self._init_db()
```

```
    def _init_db(self):
```

```
        """Initialize database tables"""
```

```
        with sqlite3.connect(self.db_path) as conn:
```

```
            cursor = conn.cursor()
```

```
            cursor.execute("""
```

```
                CREATE TABLE IF NOT EXISTS appointments (
```

```
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
                    date TEXT NOT NULL,
```

```
                    time TEXT NOT NULL,
```

```
                    location TEXT NOT NULL,
```

```
                    checklist TEXT NOT NULL,
```

```
                    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```

        synced BOOLEAN DEFAULT 0
    )
    """)
conn.commit()

```

```
def save_appointment(self, appointment: Appointment) -> int:
```

```
    """Save appointment locally and return ID"""
```

```
    with sqlite3.connect(self.db_path) as conn:
```

```
        cursor = conn.cursor()
```

```
        cursor.execute("""
```

```
            INSERT INTO appointments (date, time, location, checklist)
```

```
            VALUES (?, ?, ?, ?)
```

```
        """, (appointment.date, appointment.time, appointment.location,
```

```
            json.dumps(appointment.checklist)))
```

```
        conn.commit()
```

```
        return cursor.lastrowid
```

```
def get_appointments_on_date(self, date: str) -> List[Appointment]:
```

```
    """Get all appointments for a specific date"""
```

```
    with sqlite3.connect(self.db_path) as conn:
```

```
        cursor = conn.cursor()
```

```
        cursor.execute("""
```

```
            SELECT id, date, time, location, checklist
```

```
            FROM appointments
```

```
            WHERE date = ?
```

```
        """, (date,))
```

```
        rows = cursor.fetchall()
```

```
        return [
```

```
            Appointment(
```

```
                appointment_id=row[0],
```

```
                date=row[1],
```

```
                time=row[2],
```

```
                location=row[3],
```

```
                checklist=json.loads(row[4])
```

```
            )
```

```
        for row in rows
```



```
]
```

```
def get_appointment_by_time(self, date: str, time: str) -> Optional[Appointment]:
```

```
    """Check if an appointment exists at specific date/time"""
```

```
    with sqlite3.connect(self.db_path) as conn:
```

```
        cursor = conn.cursor()
```

```
        cursor.execute("""
```

```
            SELECT id, date, time, location, checklist
```

```
            FROM appointments
```

```
            WHERE date = ? AND time = ?
```

```
        """, (date, time))
```

```
        row = cursor.fetchone()
```

```
        if row:
```

```
            return Appointment(
```

```
                appointment_id=row[0],
```

```
                date=row[1],
```

```
                time=row[2],
```

```
                location=row[3],
```

```
                checklist=json.loads(row[4])
```

```
            )
```

```
        return None
```

```
def mark_as_synced(self, appointment_id: int):
```

```
    """Mark appointment as synced with cloud"""
```

```
    with sqlite3.connect(self.db_path) as conn:
```

```
        cursor = conn.cursor()
```

```
        cursor.execute("""
```

```
            UPDATE appointments SET synced = 1
```

```
            WHERE id = ?
```

```
        """, (appointment_id,))
```

```
        conn.commit()
```

```
def get_unsynced_appointments(self) -> List[Appointment]:
```

```
    """Get appointments that haven't been synced yet"""
```

```
    with sqlite3.connect(self.db_path) as conn:
```

```
        cursor = conn.cursor()
```

```

cursor.execute("""
    SELECT id, date, time, location, checklist
    FROM appointments
    WHERE synced = 0
""")
rows = cursor.fetchall()
return [
    Appointment(
        appointment_id=row[0],
        date=row[1],
        time=row[2],
        location=row[3],
        checklist=json.loads(row[4])
    )
    for row in rows
]

```

```

# =====
# CLOUD SYNC SERVICE
# =====

```

```

class CloudSyncService:

```

```

    """Handles cross-device sync via HTTPS"""

```

```

    def __init__(self, api_base_url: str, api_key: str = None):

```

```

        self.api_base_url = api_base_url.rstrip('/')

```

```

        self.api_key = api_key

```

```

        self.session = requests.Session()

```

```

        if api_key:

```

```

            self.session.headers.update({'Authorization': f'Bearer {api_key}'})

```

```

        self.session.headers.update({'Content-Type': 'application/json'})

```

```

    def sync_appointment(self, appointment: Appointment) -> bool:

```

```

        """Sync appointment to cloud"""

```

```

        try:

```

```

response = self.session.post(
    f'{self.api_base_url}/appointments',
    json={
        'date': appointment.date,
        'time': appointment.time,
        'location': appointment.location,
        'checklist': appointment.checklist
    },
    timeout=10
)
response.raise_for_status()
return True
except requests.RequestException as e:
    print(f'Sync failed: {e}')
    return False

```

```

def get_appointments(self, user_id: str = None) -> List[Dict]:
    """Fetch appointments from cloud (for cross-device sync)"""
    try:
        params = {'user_id': user_id} if user_id else {}
        response = self.session.get(
            f'{self.api_base_url}/appointments',
            params=params,
            timeout=10
        )
        response.raise_for_status()
        return response.json().get('appointments', [])
    except requests.RequestException as e:
        print(f'Fetch failed: {e}')
        return []

```

```

# =====
# ALERT SCHEDULER
# =====

```

```

class AlertScheduler:
    """Handles scheduling of email, SMS, and push alerts"""

    def __init__(self, email_config: Dict = None, sms_gateway: str = None):
        self.email_config = email_config or {}
        self.sms_gateway = sms_gateway
        self.scheduled_alerts: List[Dict] = []

    def schedule_alert(self, appointment: Appointment, alert_config: AlertConfig) -> bool:
        """Schedule an alert for an appointment"""
        try:
            # Parse appointment datetime
            appt_datetime = datetime.strptime(
                f"{appointment.date} {appointment.time}",
                "%Y-%m-%d %H:%M"
            )
            alert_time = appt_datetime - timedelta(minutes=alert_config.minutes_before)

            alert = {
                'appointment_id': appointment.appointment_id,
                'alert_type': alert_config.alert_type.value,
                'recipient': alert_config.recipient,
                'scheduled_for': alert_time.isoformat(),
                'appointment_details': {
                    'date': appointment.date,
                    'time': appointment.time,
                    'location': appointment.location
                }
            }
            self.scheduled_alerts.append(alert)
            print(f"Alert scheduled: {alert_config.alert_type.value} to {alert_config.recipient} at {alert_time}")
            return True
        except Exception as e:
            print(f"Failed to schedule alert: {e}")
            return False

```

```

def send_email_alert(self, recipient: str, appointment: Dict) -> bool:
    """Send email alert using SMTP"""
    if not self.email_config:
        print("Email not configured")
        return False

    try:
        subject = f"Appointment Reminder: {appointment['date']} at {appointment['time']}"
        body = f"""
Appointment Reminder

Date: {appointment['date']}
Time: {appointment['time']}
Location: {appointment['location']}

Please ensure you have your checklist ready.
"""

        context = ssl.create_default_context()
        with smtplib.SMTP(self.email_config.get('smtp_server', 'smtp.gmail.com'),
                          self.email_config.get('smtp_port', 587)) as server:
            server.starttls(context=context)
            server.login(self.email_config['username'], self.email_config['password'])
            message = f"Subject: {subject}\n\n{body}"
            server.sendmail(self.email_config['username'], recipient, message)
            print(f"Email alert sent to {recipient}")
            return True
    except Exception as e:
        print(f"Email failed: {e}")
        return False

def send_sms_alert(self, phone_number: str, appointment: Dict) -> bool:
    """Send SMS alert (simplified - would use Twilio or similar in production)"""
    if not self.sms_gateway:
        print("SMS gateway not configured")

```

```
return False
```

```
message = f'Reminder: {appointment['date']} {appointment['time']} at  
{appointment['location']}'  
print(f'SMS alert sent to {phone_number}: {message}')  
# In production: integrate with Twilio, AWS SNS, or other SMS provider  
return True
```

```
def send_push_alert(self, device_token: str, appointment: Dict) -> bool:  
    """Send push notification (simplified - would use FCM/APNS in production)"""  
    message = {  
        'title': 'Appointment Reminder',  
        'body': f'{appointment['date']} at {appointment['time']} - {appointment['location']}'  
    }  
    print(f'Push alert sent to {device_token}: {message}')  
    # In production: integrate with Firebase Cloud Messaging or Apple Push Notification  
    Service  
    return True
```

```
def process_pending_alerts(self):  
    """Process and send any pending alerts"""  
    now = datetime.now()  
    remaining_alerts = []  
  
    for alert in self.scheduled_alerts:  
        alert_time = datetime.fromisoformat(alert['scheduled_for'])  
        if alert_time <= now:  
            # Send the alert  
            if alert['alert_type'] == 'email':  
                self.send_email_alert(alert['recipient'], alert['appointment_details'])  
            elif alert['alert_type'] == 'sms':  
                self.send_sms_alert(alert['recipient'], alert['appointment_details'])  
            elif alert['alert_type'] == 'push':  
                self.send_push_alert(alert['recipient'], alert['appointment_details'])  
            # Don't keep in list after sending  
        else:
```

```

        remaining_alerts.append(alert)

self.scheduled_alerts = remaining_alerts

# =====
# APPOINTMENT MANAGER (MAIN CONTROLLER)
# =====

class AppointmentManager:
    """Main controller for appointment management"""

    def __init__(self, cloud_sync_service: CloudSyncService = None,
                  alert_scheduler: AlertScheduler = None):
        self.db = LocalDatabase()
        self.cloud_sync = cloud_sync_service
        self.alert_scheduler = alert_scheduler or AlertScheduler()

    def validate_appointment_input(self, date: str, time: str, location: str,
                                   checklist: List[str]) -> Dict[str, Any]:
        """
        Validate input data formatting
        Returns: {'valid': bool, 'errors': List[str]}
        """
        errors = []

        # Validate date format
        try:
            datetime.strptime(date, "%Y-%m-%d")
        except ValueError:
            errors.append("Date must be in YYYY-MM-DD format")

        # Validate time format
        try:
            datetime.strptime(time, "%H:%M")
        except ValueError:

```

```

        errors.append("Time must be in HH:MM format (24-hour)")

# Validate location
if not location or len(location.strip()) < 2:
    errors.append("Location must be at least 2 characters")

# Validate checklist (can be empty, but must be list)
if not isinstance(checklist, list):
    errors.append("Checklist must be a list")

return {
    'valid': len(errors) == 0,
    'errors': errors
}

def check_time_slot_available(self, date: str, time: str) -> bool:
    """
    Query existing appointments for time slot conflict
    Returns True if available, False if conflict
    """
    existing = self.db.get_appointment_by_time(date, time)
    return existing is None

def save_appointment(self, date: str, time: str, location: str,
                     checklist: List[str]) -> Dict[str, Any]:
    """
    Save appointment locally
    Returns: {'success': bool, 'appointment_id': int, 'message': str}
    """
    try:
        appointment = Appointment(
            date=date,
            time=time,
            location=location,
            checklist=checklist
        )

```



```

        appointment_id = self.db.save_appointment(appointment)
        appointment.appointment_id = appointment_id

    return {
        'success': True,
        'appointment_id': appointment_id,
        'appointment': appointment,
        'message': "Appointment saved locally"
    }
except Exception as e:
    return {
        'success': False,
        'appointment_id': None,
        'message': f"Save failed: {e}"
    }

def confirm_local_save(self, appointment_id: int) -> bool:
    """Confirm that appointment was saved locally"""
    # Verify appointment exists in database
    with sqlite3.connect(self.db.db_path) as conn:
        cursor = conn.cursor()
        cursor.execute("SELECT id FROM appointments WHERE id = ?", (appointment_id,))
        return cursor.fetchone() is not None

def sync_across_devices(self, appointment: Appointment) -> bool:
    """Sync appointment across devices via HTTPS"""
    if not self.cloud_sync:
        print("Cloud sync not configured - skipping")
        return False

    success = self.cloud_sync.sync_appointment(appointment)
    if success:
        self.db.mark_as_synced(appointment.appointment_id)
        print(f"Appointment {appointment.appointment_id} synced to cloud")
    return success

```

```
def schedule_alerts(self, appointment: Appointment, alert_configs: List[AlertConfig]) -> List[bool]:
```

```
    """Schedule alerts based on FR04"""
```

```
    results = []
```

```
    for config in alert_configs:
```

```
        config.appointment_id = appointment.appointment_id
```

```
        success = self.alert_scheduler.schedule_alert(appointment, config)
```

```
        results.append(success)
```

```
    return results
```

```
def display_success_message(self, appointment: Appointment) -> str:
```

```
    """Generate success message for UI"""
```

```
    return f"Appointment created successfully!\nDate: {appointment.date}\nTime: {appointment.time}\nLocation: {appointment.location}\nChecklist: {''.join(appointment.checklist)}"
```

```
def update_overview(self) -> List[Appointment]:
```

```
    """Get all appointments for overview display (FR01)"""
```

```
    # Get appointments from local DB
```

```
    with sqlite3.connect(self.db.db_path) as conn:
```

```
        cursor = conn.cursor()
```

```
        cursor.execute("""
```

```
            SELECT id, date, time, location, checklist
```

```
            FROM appointments
```

```
            ORDER BY date, time
```

```
        """)
```

```
    rows = cursor.fetchall()
```

```
    appointments = [
```

```
        Appointment(
```

```
            appointment_id=row[0],
```

```
            date=row[1],
```

```
            time=row[2],
```

```
            location=row[3],
```

```
            checklist=json.loads(row[4])
```

```
        )
```

```
    for row in rows
```

```
]
return appointments
```

```
# =====
# MAIN SEQUENCE (Based on the Diagram)
# =====
```

```
class AppointmentController:
```

```
    """
```

```
    Implements the exact sequence from the diagram:
```

1. Customer taps "Add Appointment" (Plus Icon)
2. Display appointment form
3. Input details & tap Save
4. Submit new appointment data
5. Validate input data formatting
6. Query existing appointments for time slot
7. ALT [Double-Booking Detected] OR [Time Slot Available]
8. Save appointment locally
9. Confirm local save
10. Sync appointment across devices (HTTPS)
11. Schedule alerts (Email, SMS, Push based on FR04)
12. Display success message & update Overview (FR01)

```
    """
```

```
def __init__(self, cloud_api_url: str = None, api_key: str = None,
              email_config: Dict = None, sms_gateway: str = None):
```

```
    # Initialize services
```

```
    self.cloud_sync = CloudSyncService(cloud_api_url, api_key) if cloud_api_url else None
```

```
    self.alert_scheduler = AlertScheduler(email_config, sms_gateway)
```

```
    self.manager = AppointmentManager(self.cloud_sync, self.alert_scheduler)
```

```
def add_appointment(self, date: str, time: str, location: str,
                    checklist: List[str],
                    alert_configs: List[AlertConfig] = None) -> Dict[str, Any]:
```

```
"""
```

Complete flow for adding an appointment following the sequence diagram

```
"""
```

```
# Step 1 & 2: Customer taps "Add Appointment" -> Display form
```

```
print("✓ Step 1-2: Add Appointment form displayed")
```

```
# Step 3: Input details & tap Save
```

```
print(f"✓ Step 3: Input received - Date: {date}, Time: {time}, Location: {location}")
```

```
# Step 4: Submit new appointment data
```

```
print("✓ Step 4: Appointment data submitted")
```

```
# Step 5: Validate input data formatting
```

```
validation = self.manager.validate_appointment_input(date, time, location, checklist)
```

```
if not validation['valid']:
```

```
    return {
```

```
        'success': False,
```

```
        'step': 'validation',
```

```
        'message': f"Validation failed: {validation['errors']}"
```

```
    }
```

```
print("✓ Step 5: Input validation passed")
```

```
# Step 6: Query existing appointments for time slot
```

```
slot_available = self.manager.check_time_slot_available(date, time)
```

```
# Step 7: ALT [Double-Booking Detected] OR [Time Slot Available]
```

```
if not slot_available:
```

```
    # Double-booking detected path
```

```
    print("⚠ Step 7: Double-booking detected - Time slot conflict")
```

```
    return {
```

```
        'success': False,
```

```
        'step': 'conflict',
```

```
        'message': f"Time slot {date} {time} is already booked. Please modify time or confirm  
override.",
```

```
        'requires_override_confirmation': True
```

```

    }

    print("✓ Step 7: Time slot available - No conflict found")

    # Step 8: Save appointment locally
    save_result = self.manager.save_appointment(date, time, location, checklist)
    if not save_result['success']:
        return {
            'success': False,
            'step': 'save',
            'message': save_result['message']
        }
    appointment = save_result['appointment']
    print(f"✓ Step 8: Appointment saved locally with ID: {appointment.appointment_id}")

    # Step 9: Confirm local save
    if self.manager.confirm_local_save(appointment.appointment_id):
        print("✓ Step 9: Local save confirmed")
    else:
        return {
            'success': False,
            'step': 'confirm_save',
            'message': "Local save could not be confirmed"
        }

    # Step 10: Sync appointment across devices (HTTPS)
    if self.cloud_sync:
        sync_success = self.manager.sync_across_devices(appointment)
        if sync_success:
            print("✓ Step 10: Sync success confirmation received")
        else:
            print("⚠ Step 10: Sync failed - will retry later")
    else:
        print("✓ Step 10: Cloud sync skipped (not configured)")

    # Step 11: Schedule alerts (Email, SMS, Push based on FR04)

```

```

if alert_configs:
    alert_results = self.manager.schedule_alerts(appointment, alert_configs)
    print(f"✓ Step 11: Alerts scheduled - Results: {alert_results}")
else:
    print("✓ Step 11: No alerts configured")

# Step 12: Display success message & update Overview (FR01)
success_message = self.manager.display_success_message(appointment)
overview = self.manager.update_overview()
print(f"✓ Step 12: {success_message}")
print(f"✓ Overview updated - Total appointments: {len(overview)}")

return {
    'success': True,
    'appointment_id': appointment.appointment_id,
    'appointment': {
        'date': appointment.date,
        'time': appointment.time,
        'location': appointment.location,
        'checklist': appointment.checklist
    },
    'message': success_message,
    'overview': [a.to_dict() for a in overview]
}

```

```

# =====
# EXAMPLE USAGE
# =====

```

```

if __name__ == "__main__":

```

```

    # Initialize the controller (can pass cloud API URL for real sync)
    controller = AppointmentController(
        cloud_api_url=None, # Set to "https://api.example.com" for real cloud sync
        api_key=None,

```

```

email_config={
    'smtp_server': 'smtp.gmail.com',
    'smtp_port': 587,
    'username': 'alerts@example.com',
    'password': 'your_password'
} if False else None, # Disable email for demo
sms_gateway=None
)

# Example: Add an appointment
print("=" * 60)
print("ADDING APPOINTMENT")
print("=" * 60)

result = controller.add_appointment(
    date="2026-04-25",
    time="14:30",
    location="Dr. Smith's Office - Room 204",
    checklist=["Insurance card", "ID", "List of medications", "Previous test results"],
    alert_configs=[
        AlertConfig(
            appointment_id=0, # Will be set by scheduler
            alert_type=AlertType.EMAIL,
            recipient="patient@example.com",
            minutes_before=60
        ),
        AlertConfig(
            appointment_id=0,
            alert_type=AlertType.SMS,
            recipient="+1234567890",
            minutes_before=30
        )
    ]
)

print("\n" + "=" * 60)

```

```

print("RESULT")
print("=" * 60)
print(f'Success: {result["success"]}')
if result['success']:
    print(f'Message: {result["message"]}')
    print(f'Appointment ID: {result["appointment_id"]}')
    print(f'Total appointments in overview: {len(result.get('overview', []))}')
else:
    print(f'Error at step {result.get('step')}: {result.get('message')}')

# Test double-booking scenario
print("\n" + "=" * 60)
print("TESTING DOUBLE-BOOKING SCENARIO")
print("=" * 60)

result2 = controller.add_appointment(
    date="2026-04-25",
    time="14:30", # Same time as previous
    location="Different Location",
    checklist=["Test"]
)

print(f'Success: {result2["success"]}')
if not result2['success']:
    print(f'Expected conflict detected: {result2.get('message')}')

```

Party Analysis

The party analysis pattern is an analysis pattern created by Martin Fowler, citing the similarities between an organisation and a person, claiming that the supertype of the two would be a “party”. This would usually be used for companies, using one analysis pattern for departments, individuals and teams. However our app, while it could potentially be used by companies, is meant to be used on a personal level by individuals. Meaning there would be no need for a “party” supertype, as the only singular users, individuals would be using the app, there would be no need for

implementing organizational or team use as it doesn't fit the original vision for the app. With that, replacing our UML would make less sense as the party analysis pattern doesn't properly help with implementing our vision for the app.